

Informe Final: Clasificador de Letras ASL

Esteban, Ernesto, Hugo

Proyecto: Clasificador de Letras ASL

Seleccionamos una submuestra del dataset original de Kaggle para este laboratorio. Tomamos 600 imágenes por cada clase, suficiente para capturar la variabilidad de los datos sin sobrepasar las limitaciones de cómputo y tiempo del entorno de trabajo.

Ejemplos de imágenes

Mostramos ejemplos de cinco letras distintas del alfabeto ASL para observar la variabilidad de las imágenes dentro de una misma clase.

Dentro de cada letra hay cambios importantes en la iluminación y el tipo de fondo. También hay variaciones en la posición y el ángulo de la mano, y en la distancia de la mano a la cámara.

Análisis exploratorio

Revisamos una muestra de imágenes de cada una de las 29 clases y todas tienen la misma resolución de 200x200 píxeles, están en formato JPEG y son a color con los tres canales RGB. No hay imágenes en escala de grises ni con dimensiones distintas dentro de la submuestra.

Las 29 clases quedaron con exactamente 600 imágenes cada una, el mínimo y el máximo por clase coinciden. El dataset original tenía suficientes imágenes disponibles por letra y la submuestra queda perfectamente balanceada.

Las letras M, N y S se parecen porque las tres se forman con el puño cerrado y el pulgar escondido debajo de los otros dedos, la diferencia entre ellas es solo la posición exacta del pulgar respecto a los dedos que lo cubren. Algo similar pasa con U, V y R, que se hacen con dos dedos extendidos y varían únicamente en si están juntos, separados o cruzados.

Para el modelo esto puede ser un problema porque son diferencias muy pequeñas en la imagen, casi a nivel de píxeles, y se pueden perder fácilmente con variaciones de ángulo, iluminación o

resolución. Es probable que estas letras generen más confusión en la clasificación que otras con formas de mano claramente distintas.

La imagen promedio se construye sumando pixel por pixel todas las fotos de una clase y dividiendo entre el total de imágenes. Si la seña se hace siempre en una posición y encuadre parecidos, el promedio sale nítido, con la forma de la mano reconocible. Si hay mucha variación de posición dentro de la clase, el promedio se ve borroso porque los bordes de la mano caen en lugares distintos de cada foto.

En las cuatro clases pasó lo segundo. El fondo, la pared y el borde de la puerta se ven casi idénticos entre A, B, M y S, lo que indica que las fotos se tomaron con la cámara fija en la misma posición. La mano en sí se pierde en una mancha difusa color piel en el centro de la imagen, sin bordes definidos, y esto pasa igual de fuerte en las cuatro letras, no solo en el grupo M y S que ya marcamos como confuso. Promediar pixeles crudos no sirve para distinguir señas, porque la posición de la mano varía lo suficiente dentro de cada clase como para que el promedio no conserve la forma. El modelo va a necesitar aprender patrones espaciales locales en vez de depender de estadísticas simples de pixel.

Tomamos el brillo promedio de pixel de cada imagen como métrica simple y comparamos su distribución entre seis clases: cuatro letras (A, B, M, S) más las clases especiales space y nothing, que no corresponden a una letra sino a espacio y ausencia de seña.

A, B, M, S y space quedan en rangos de brillo muy parecidos entre sí, con medias entre 126.2 y 136.8 y desviaciones de 18.3 a 20.6. La clase nothing es la que se distingue: su media sube a 146.9 y su desviación llega a 29.5. Tiene sentido, porque nothing captura escenas sin mano en el encuadre, así que queda más expuesta a cambios de iluminación de fondo y menos cubierta por la sombra que genera la propia mano cuando está presente. Para las letras entre sí el brillo no aporta mucho, porque la iluminación depende más de la sesión de captura que de la seña específica.

Tomamos una imagen representativa de M y otra de S y graficamos la intensidad de cada canal de color por separado. En ambas letras los tres canales, rojo, verde y azul, siguen prácticamente la misma forma de distribución, solo desplazados un poco entre sí, lo cual es normal en piel bajo luz interior. No hay un canal que se dispare independiente de los otros.

Esto importa para la decisión de color contra escala de grises que documentamos más adelante en preprocesamiento, porque si los tres canales se mueven juntos, convertir a escala de grises no debería perder información relevante para distinguir la forma de la mano. La distribución en sí varía de imagen a imagen según el fondo y la sombra de esa foto puntual, como se ve en que M tiene un pico más marcado cerca de 200 y S tiene la masa más repartida entre valores bajos y altos, pero eso depende de la escena capturada en esa imagen particular, no de un patrón sistemático de la letra.

Split de entrenamiento, validación y prueba

El set de prueba que trae Kaggle para este dataset viene con muy pocas imágenes, pensado más para una demo que para evaluar un modelo con seriedad. Por eso armamos nuestro propio split a partir de los datos de entrenamiento que ya tenemos descargados, en vez de depender de ese set oficial.

Usamos una proporción 70 por ciento entrenamiento, 15 por ciento validación y 15 por ciento prueba, estratificada por clase para que las 29 letras mantengan la misma proporción en los tres conjuntos. Estratificar es importante acá porque aunque el dataset está balanceado en total, un split aleatorio simple podría dejar alguna clase con pocas imágenes en validación o prueba solo por azar, y eso haría más ruidosa la evaluación de esa clase en particular. Con 600 imágenes por clase, un 15 por ciento son 90 imágenes de prueba por letra, suficiente para medir el desempeño sin sacrificar demasiados datos de entrenamiento.

Con las 17400 imágenes de la submuestra, el split queda en 12180 para entrenamiento, 2610 para validación y 2610 para prueba, y la clase A aporta exactamente 90 imágenes al set de prueba, igual que cada una de las 29 clases. Guardamos las listas de rutas de archivo y etiquetas de cada conjunto para usarlas directamente en el pipeline de entrenamiento.

Preprocesamiento

Antes de entrenar cualquier modelo hay que dejar las 17400 imágenes en un formato manejable para el hardware disponible en el laboratorio. Las imágenes originales vienen a 200x200 en RGB, lo que da 40000 píxeles por canal y 120000 valores por imagen. Entrenar varias arquitecturas distintas sobre eso, incluyendo un Random Forest que no tiene forma de compartir cómputo entre píxeles como sí lo hace una convolución, sale caro en tiempo.

Por eso reducimos cada imagen a 64x64 antes de meterla a cualquier modelo. Esto deja 4096 píxeles por canal, casi 10 veces menos que el original, y sigue siendo suficiente resolución para que se distinga la forma general de la mano y la posición de los dedos, que es la información que separa una letra de otra. El detalle fino que se pierde al reducir, como la textura de la piel o pequeñas variaciones de sombra, no aporta a la tarea de clasificación y hasta podría hacer que el modelo se sobreajuste a detalles irrelevantes de cada foto.

Para la normalización dividimos cada pixel entre 255 para dejarlo en el rango de 0 a 1, en vez de estandarizar restando la media y dividiendo entre la desviación estándar. En la imagen de ejemplo el rango pasa de 0 a 255 a quedar entre 0.027 y 1.0. Con ese rango alcanza para este caso porque las imágenes ya vienen en una escala de iluminación bastante uniforme, según la distribución de brillo por clase, así que no hace falta el paso extra de calcular media y desviación por canal sobre todo el set de entrenamiento.

Mantenemos las imágenes a color en vez de pasarlas a escala de grises. Un solo canal ahorraría memoria, pero en el histograma de intensidad de M y S los tres canales se mueven prácticamente

juntos, lo que confirma que no hay señal de color que separe las letras entre sí, la información está en la forma de la mano y no en el tono. Dejamos los tres canales porque el costo de cómputo ya se resolvió con la reducción a 64x64, y quitar canales en este punto sería una optimización que no hace falta para el tamaño de este laboratorio.

No aplicamos ningún filtro adicional de suavizado o realce de bordes. Las fotos ya vienen limpias, tomadas con una cámara fija y sin ruido visible, y el problema real que encontramos en el análisis exploratorio, la confusión entre M, N, S y entre U, V, R, es de forma de mano y no de calidad de imagen, así que un filtro no ayudaría ahí. El único preprocesamiento que aplicamos es resize a 64x64 y la normalización a 0 y 1.

Selección de modelos

Entrenamos y comparamos cuatro modelos sobre las imágenes ya reducidas a 64x64 en color: dos redes convolucionales con distinta profundidad, una red totalmente conectada como referencia sin convoluciones, y un algoritmo clásico de machine learning como punto de comparación fuera del mundo de redes neuronales.

La primera CNN es una arquitectura chica, pensada como línea base. Tiene tres bloques de convolución con 32, 64 y 128 filtros respectivamente, cada uno seguido de max pooling para ir reduciendo el tamaño espacial, y termina en una capa densa de 128 neuronas antes de la salida de 29 clases con softmax. La segunda CNN agrega un cuarto bloque convolucional con 256 filtros y normalización por lotes después de cada convolución, además de dropout antes de la capa densa final. La idea es medir si la profundidad y la regularización extra realmente ayudan con este dataset o si con este tamaño de submuestra ya alcanza la arquitectura más simple, sobre todo pensando en las letras que se confunden entre sí como M, N, S y U, V, R, donde una red más profunda podría captar mejor las diferencias sutiles de posición del pulgar o los dedos.

Como referencia sin convoluciones entrenamos una red totalmente conectada que recibe la imagen aplanada, 64 por 64 por 3 valores de entrada, pasa por dos capas densas de 256 y 128 neuronas y termina en la salida de 29 clases. Esta red no puede aprovechar que los píxeles cercanos están relacionados espacialmente, así que sirve para confirmar si la estructura convolucional realmente aporta algo sobre este problema o si una red densa ya es suficiente.

Para el algoritmo alterno elegimos Random Forest en vez de SVM o KNN. Con 12180 imágenes de entrenamiento y vectores de más de 12000 valores por imagen, un KNN sale caro porque tiene que calcular distancia contra todo el set de entrenamiento en cada predicción, y un SVM con kernel no lineal escala mal en tiempo de entrenamiento con este volumen de datos. Random Forest entrena razonablemente rápido incluso con muchas variables, no necesita que los datos estén escalados de una forma particular, y de paso da una medida de importancia por pixel que sirve para entender en qué zonas de la imagen se está fijando el modelo, algo que se puede comparar contra lo que aprenden las CNN.

Plan de procesamiento de imágenes

Resumimos acá todo el camino que recorre una imagen desde el dataset crudo de Kaggle hasta quedar lista para entrar a cualquiera de los modelos.

1. Dataset crudo: el dataset original de Kaggle trae miles de imágenes por letra. De ahí tomamos una submuestra de 600 imágenes por clase con semilla fija en 42, que es la que vive en la carpeta data de este proyecto y con la que se trabajó todo el análisis exploratorio.
2. División en conjuntos: sobre esas 17400 imágenes aplicamos el split estratificado 70 por ciento entrenamiento, 15 por ciento validación y 15 por ciento prueba definido antes, para que ningún conjunto quede sesgado hacia unas letras más que otras.
3. Resize a 64x64: cada imagen pasa de 200x200 a 64x64 para bajar el costo de cómputo sin perder la forma general de la mano.
4. Normalización: dividimos los valores de pixel entre 255 para dejarlos en el rango de 0 a 1, que es el formato que esperan las capas de entrada de las redes.
5. Aumentos de datos, solo sobre el conjunto de entrenamiento: acá hay que tener cuidado porque no todos los aumentos típicos de visión por computadora tienen sentido para señas. Aplicamos rotaciones leves, de unos 10 a 15 grados como máximo, pequeños corrimientos de posición y zoom leve, y variaciones menores de brillo y contraste, para simular la variabilidad de ángulo y distancia a la cámara que ya vimos en el análisis exploratorio. Lo que evitamos por completo es el flip horizontal, porque muchas señas del alfabeto dependen de qué mano y en qué orientación se hacen, y voltear la imagen puede convertir una seña válida en una que no corresponde a esa letra o directamente en otra letra distinta. Por la misma razón tampoco aplicamos flip vertical ni rotaciones grandes que puedan hacer que una seña se parezca a otra, como podría pasar entre M, N y S si se rota demasiado.
6. Conversión a tensores: ya con las imágenes en 64x64, normalizadas y aumentadas cuando corresponde, las convertimos a tensores. Para las CNN y la red totalmente conectada esto es un tensor de forma 64 por 64 por 3 por imagen. Para el Random Forest aplanamos ese mismo tensor a un vector de una dimensión, porque ese modelo no trabaja con estructura espacial como sí lo hacen las redes.

Ejercicio 4: modelos CNN

Entrenamos acá las dos arquitecturas convolucionales descritas en la sección de selección de modelos, reutilizando el split 70/15/15 y el resize a 64x64 con normalización que ya definimos. Para cada una probamos primero un par de configuraciones de entrenamiento distintas sobre pocas épocas para no perder tiempo entrenando algo mal ajustado, y con la configuración que mejor le va seguimos entrenando más épocas con paro temprano hasta que la red deja de mejorar en validación.

De las tres configuraciones que probamos, la que ganó fue learning rate de 0.001 con batch de 32, con 0.9395 de accuracy en validación tras 6 épocas, arriba del 0.9000 y el 0.8816 de las otras dos combinaciones. Con esa configuración entrenamos más épocas y el paro temprano cortó en la época 10 de las 18 que dejamos como tope. La red llegó a 0.9467 de accuracy en el set de prueba, con una pérdida de 0.1858.

La matriz de confusión coincide con lo que anotamos en el análisis exploratorio. Los dos pares que más se confunden son V con U, 10 imágenes, y V con W, 9, ambos dentro del grupo de dos dedos extendidos que marcamos como visualmente parecido. Después vienen del con Q, 8 imágenes, y K con W, 6. Ninguno pasa de 10 imágenes mal clasificadas sobre 90 posibles por clase, así que la confusión existe pero no domina el resultado.

CNN profunda

La CNN profunda quedó por debajo de la base. En el sweep, la configuración con learning rate de 0.001 colapsó a 0.0345 de accuracy en validación, cerca del nivel de adivinar al azar entre 29 clases, y la de 0.0005 se quedó en 0.3870 tras 6 épocas. Con esa segunda configuración entrenamos las 18 épocas completas sin que el paro temprano cortara antes, y la accuracy de entrenamiento terminó en 0.5309, mientras que la de validación fue inestable, subió a 0.9073 en la época 14 y cerró en 0.8858. La accuracy en prueba quedó en 0.8736, contra 0.9467 de la CNN base.

La explicación tiene dos partes. Agregar el cuarto bloque convolucional y la normalización por lotes hace la red más sensible al learning rate, como se ve en que 0.001 la hizo colapsar mientras que a la CNN base ese mismo valor le funcionó bien. Además la normalización por lotes se comporta distinto en modo entrenamiento, donde usa estadísticas del batch actual, que en modo evaluación, donde usa el promedio acumulado, y eso explica por qué la accuracy de entrenamiento se ve más baja que la de validación durante todo el entrenamiento. La matriz de confusión lo refleja, con U y R llegando a 68 imágenes mal clasificadas y G con H a 41, muy por encima de cualquier par de la CNN base. Con el presupuesto de épocas que usamos, la arquitectura más profunda no terminó de converger.

Comparacion y seleccion de la mejor CNN

Nos quedamos con la CNN base para el resto de comparaciones. La diferencia entre 0.9467 y 0.8736 de accuracy en prueba es grande, y tiene que ver con que la arquitectura más profunda necesitaba más tiempo de entrenamiento del que le dimos. Con más épocas o un learning rate con calentamiento gradual la CNN profunda podría terminar superando a la base, pero dentro del presupuesto de cómputo de este laboratorio la base es la que funciona.

Ejercicio 5: red neuronal simple

Entrenamos ahora la red totalmente conectada que describimos en la selección de modelos, sobre las mismas imágenes 64x64 en color, para comparar contra la mejor CNN.

La red totalmente conectada llegó a 0.6556 de accuracy en prueba, muy por debajo de la CNN base y también de la CNN profunda. A diferencia de las CNN, no llegó a estabilizarse en 18 épocas: la accuracy de entrenamiento seguía subiendo al final, 0.6905, y la de validación cerró en 0.6498 sin señales de que el paro temprano fuera a cortar pronto.

La matriz de confusión es lo más revelador. Los pares que más confunde son O con M, 32 imágenes, V con U, 25, y V con W, 24, así que mezcla letras que sí marcamos como parecidas con otras que no tienen ninguna relación visual, como O con M. Esto encaja con lo que vimos en la imagen promedio por clase, donde la posición de la mano varía lo suficiente dentro de cada clase como para que un pixel en una posición fija no signifique lo mismo de una foto a otra. La red totalmente conectada trata cada pixel como una entrada independiente y no tiene forma de reconocer que un patrón de bordes es el mismo sin importar en qué parte de la imagen aparece, así que termina confundiendo letras por razones que no tienen que ver con el parecido real de la seña. Las CNN sí tienen esa invariancia espacial gracias a las convoluciones, y por eso les va mejor incluso cuando, como la CNN profunda, no terminan de converger.

Ejercicio 6: algoritmo alternativo, Random Forest

Ya justificamos en la selección de modelos por qué elegimos Random Forest sobre SVM o KNN para este problema: con miles de imágenes y vectores de más de 12000 valores, KNN sale caro en tiempo de predicción y SVM con kernel no lineal escala mal en tiempo de entrenamiento. Acá lo entrenamos sobre las imágenes aplanadas y vemos los resultados.

Con 200 árboles y sin límite de profundidad, el Random Forest llegó a 0.9621 de accuracy en validación y 0.9582 en prueba, ya por encima de la CNN base sin haber ajustado nada todavía. Trabaja sobre exactamente la misma representación de píxeles aplanados que la red totalmente conectada, que se quedó en 0.6556, así que la diferencia de más de 30 puntos está en el modelo y no en los datos que recibe.

Un árbol de decisión no necesita una regla global como una red densa. Cada árbol aprende reglas del tipo si el pixel en tal posición supera tal valor entonces probablemente es tal clase, y con cientos de árboles votando sobre subconjuntos distintos de píxeles el conjunto termina siendo bastante robusto. Esto conecta con la imagen promedio por clase: el fondo, la pared y el borde de la puerta salían casi idénticos entre clases porque la cámara estaba fija, así que hay píxeles de fondo consistentes que un árbol puede usar como referencia indirecta de dónde suele caer la mano, algo que una red totalmente conectada con descenso de gradiente no aprovecha igual de bien en el mismo número de épocas. La matriz de confusión sale limpia, el par que más se confunde, D con E, no pasa de 6 imágenes.

Ajuste de parametros del Random Forest

El parámetro que más movió la aguja fue la profundidad máxima, no la cantidad de árboles. Limitar la profundidad a 10 hundió la accuracy de validación a 0.9054, bastante por debajo del resto, mientras que 20 y 40 quedaron en 0.9582 y 0.9617, y las configuraciones sin límite se mantuvieron arriba de 0.957. Con 29 clases y más de 12000 variables de entrada, cada árbol necesita crecer bastante para separar las clases, así que cortar la profundidad le quita capacidad al modelo en vez de ayudarlo a generalizar.

La configuración ganadora fue 300 árboles sin límite de profundidad, con 0.9640 en validación. Entrenada sobre el set de entrenamiento completo dio 0.9598 en prueba, apenas arriba del 0.9582 del modelo base con 200 árboles. La mejora es pequeña pero real, y confirma que el Random Forest ya estaba cerca de su techo con la configuración inicial: la ganancia vino de subir el número de árboles, no de restringir el modelo.

Ejercicio 7: image augmentation

Aplicamos ahora los aumentos que dejamos definidos en el plan de procesamiento de imágenes: rotaciones leves, corrimientos de posición, zoom leve y variaciones de brillo y contraste, solo sobre el conjunto de entrenamiento. Nada de flip.

El aumento no le hizo lo mismo a los tres modelos. La CNN base subió de 0.9467 a 0.9567 de accuracy en prueba, un punto porcentual, el Random Forest pasó de 0.9598 a 0.9605, menos de un décimo de punto, y la red totalmente conectada bajó de 0.6556 a 0.6130. Durante el entrenamiento de la CNN aumentada la accuracy de entrenamiento se quedó alrededor de 0.52 mientras la de validación llegó a 0.9621, que es lo esperable: el set de entrenamiento aumentado mezcla las imágenes originales con las rotadas, corridas y con brillo alterado, más difíciles de clasificar, mientras que validación sigue siendo el set limpio de siempre.

Que la CNN sea la que más se beneficia tiene sentido porque es la única de las tres que aprende filtros espaciales, y esos filtros se vuelven más robustos viendo la misma mano en distintas posiciones y ángulos. La red totalmente conectada no tiene esa estructura para aprovechar la variedad extra, trata cada pixel como independiente, así que las variaciones no le enseñan invariancia y terminan actuando como ruido, que es justo lo que muestra su caída de 4 puntos. El Random Forest tampoco se movió, porque ya estaba cerca de su techo con esta representación de pixeles aplanados y las reglas de corte por pixel que aprende no generalizan mejor por duplicar el set con versiones desplazadas de las mismas imágenes.

Sobre por qué el flip horizontal es distinto a los demás aumentos: en las imágenes de M, N y S volteadas horizontalmente, la mano sigue pareciendo una mano, pero el orden de izquierda a derecha de los dedos y el pulgar queda invertido. Estas señas se distinguen exactamente por esa posición relativa del pulgar contra los demás dedos, y N con M aparece con 32 imágenes mal clasificadas en la CNN profunda, uno de sus pares más confundidos. Voltar la imagen no agrega una variación de cámara como sí lo hacen la rotación leve o el corrimiento de posición,

cambia la geometría misma de la seña, y podría acercar visualmente una M volteada a como luce una N o una S real, metiendo una seña falsa justo en el par que el modelo ya tiene más difícil. Rotar unos grados o mover la mano unos píxeles sigue siendo la misma seña vista con otro encuadre, mientras que el flip cambia qué seña es.

Por esto los aumentos que sí aplicamos son rotación leve, hasta 15 grados, corrimiento de posición, zoom leve y variaciones de brillo y contraste, todos aumentos que simulan variabilidad de cómo se capturó la foto y no de la forma de la mano. Rotaciones más grandes también las evitamos, porque igual que el flip podrían acercar una seña a otra dentro de los grupos M, N, S y U, V, R que identificamos desde el análisis exploratorio. El flip horizontal y vertical quedan fuera por completo, porque ninguno de los dos representa una variación real de cómo alguien haría esa seña frente a una cámara, invierten la seña misma.

Consolidacion de los ejercicios 4 a 7

Con los ocho modelos ordenados, arriba quedan el Random Forest con aumento con 0.9605, el Random Forest ajustado con 0.9598 y el Random Forest base con 0.9582, y muy cerca la CNN base con aumento con 0.9567. Estos cuatro quedan a menos de medio punto porcentual entre sí. Después viene la CNN base sin aumento con 0.9467, y ya en otro nivel la CNN profunda con 0.8736 y la red totalmente conectada, entre 0.6556 y 0.6130 según si lleva aumento o no.

Que el Random Forest termine arriba no significa que sea el mejor modelo para el problema real de clasificar señas. Buena parte de su desempeño puede venir de aprovechar píxeles de fondo consistentes entre clases, porque las fotos de este dataset se tomaron con la misma cámara fija, algo que ya se veía en la imagen promedio por clase. Ese patrón no necesariamente se sostiene con fotos nuevas tomadas en otras condiciones, mientras que la CNN aprende filtros que reconocen la forma de la mano sin depender tanto del fondo. Por eso seguimos considerando tanto el Random Forest ajustado como la CNN base con aumento como candidatos serios, en vez de descartar la CNN solo por haber quedado unas décimas abajo en este set de prueba particular.

De los cuatro ejercicios, el patrón que más se repite es que la arquitectura por sí sola no garantiza nada sin el ajuste de parámetros y sin pensar qué tipo de variación tiene sentido agregarle a los datos. La CNN profunda con más capas no le ganó a la más simple porque no tuvo suficiente entrenamiento para converger, el Random Forest mejoró más por ampliar el número de árboles que por restringir su profundidad, y el aumento de datos solo ayudó al modelo que tenía la estructura para aprovecharlo. El paso que sigue es ver qué hacen estos dos candidatos con fotos tomadas fuera del dataset.

Ejercicio 8: prueba con senas propias

Para ver si los dos candidatos generalizan fuera del dataset de Kaggle, tomamos fotos de nuestras propias manos haciendo señas del alfabeto ASL, con fondo e iluminación distintos

a los del dataset original. Antes de tomarlas generamos una hoja de referencia con imágenes reales del dataset para saber exactamente qué forma imitar.

Cargamos 22 fotos que cubren 18 letras y vienen de dos conjuntos de captura distintos. El Random Forest ajustado acertó 0 de 22 y la CNN base con aumento acertó 2 de 22, un 0.0909. Contra el 0.9598 y el 0.9567 que estos mismos modelos dieron en el set de prueba del dataset, la caída es total. Los modelos no generalizan a fotos tomadas en condiciones distintas a las del dataset original, aunque en el set de prueba se vean casi perfectos.

Los dos conjuntos de fotos se tomaron en condiciones bastante distintas y eso cambia la forma del error. En uno la mano ocupa una porción chica del cuadro y el resto es pared, techo, mueble y hombro. En el otro la mano está mucho más cerca del lente y llena buena parte del cuadro, sobre un fondo de madera más oscuro y con menos contraste. Esa diferencia de encuadre es la que mejor explica lo que hace cada modelo.

El Random Forest predijo nothing en 17 de las 22 fotos. Las 15 del conjunto con más fondo cayeron todas en nothing, sin excepción. La clase nothing del dataset son fotos de pared y techo vacíos, con el mismo tono de pared y la misma luz que aparece de fondo en nuestras fotos, y es además la clase con el brillo promedio y la desviación más altos de las que medimos en el análisis exploratorio, 146.9 y 29.5. En el dataset la mano llena casi todo el cuadro, así que cuando la proporción de fondo crece el modelo se queda sin la señal que estaba usando y la foto se le parece más a una escena vacía que a una seña. En el conjunto donde la mano está cerca del lente el comportamiento cambia: solo 2 de 7 fotos cayeron en nothing y las otras 5 recibieron letras, cuatro veces H y una vez P. Sigue siendo 0 aciertos, pero deja de contestar que no hay mano. El Random Forest no depende de un fondo específico, depende de que el fondo sea mínimo.

La CNN con aumento falló distinto y también cambió según el encuadre. En el conjunto con más fondo predijo L en 14 de 15 fotos sin importar la seña real, y su único acierto ahí fue justamente cuando la foto era una L. Ese colapso a una salida por defecto es típico de un modelo que se enfrenta a datos muy fuera de lo que vio en entrenamiento, no de un modelo que esté leyendo la forma de la mano. En el conjunto con la mano más cerca las 7 predicciones fueron 7 letras distintas, Z, H, Y, W, B, P y R, con la H acertada. Deja de repetir siempre lo mismo y empieza a reaccionar a la forma, aunque casi siempre se equivoque. El aumento que aplicamos, rotación, zoom, brillo y contraste, ayuda contra variaciones pequeñas pero no prepara al modelo para un encuadre con esta cantidad de fondo extra ni para un ángulo de cámara tan distinto al de un dataset tomado con la mano pegada al lente.

Cuatro letras, A, M, O y V, quedaron fotografiadas en los dos conjuntos, con manos distintas y en fondos e iluminaciones distintas. Ninguna de esas ocho fotos la acertó ninguno de los dos modelos. Lo que cambia no es el acierto sino el tipo de error: en las cuatro del conjunto con más fondo el Random Forest respondió nothing y la CNN respondió L, mientras que en las cuatro del otro conjunto el Random Forest respondió H en tres y nothing en la V, y la CNN respondió Z, W, B y R. La misma seña, hecha por dos manos en dos escenarios, produce errores de naturaleza distinta según las condiciones de captura y no según la letra.

El análisis se completa cuando se sumen las fotos del integrante restante y haya un tercer conjunto de condiciones para contrastar.

Ejercicio 9: accesibilidad y sesgo

Lo que salió en el ejercicio 8 muestra limitaciones fuertes de este dataset para pensar en un producto real de accesibilidad como SignBridge.

El ángulo de cámara y la distancia son el problema más evidente. Las 17400 imágenes del dataset se tomaron con la mano pegada al lente, llenando casi todo el cuadro, con un fondo fijo de pared y techo. Nuestras fotos, tomadas con celular a una distancia normal de uso, tienen mucho más fondo alrededor de la mano. Esa sola diferencia de encuadre bastó para que el Random Forest cayera a 0 de 22 y la CNN a 2 de 22, contra el 0.9598 y el 0.9567 del set de prueba. Un producto real no puede asumir que el usuario va a sostener la mano exactamente a la misma distancia y ángulo que se usó para entrenar.

Tener dos conjuntos de fotos permite separar un poco el efecto. En el conjunto donde la mano ocupa poca parte del cuadro, el Random Forest respondió nothing en las 15 fotos y la CNN respondió L en 14 de 15. En el conjunto donde la mano está cerca del lente, el Random Forest dejó de responder nothing en 5 de 7 y la CNN dio 7 letras distintas en vez de repetir una sola, con la accuracy subiendo de 0.0667 a 0.1429. Ninguno de los dos se vuelve usable, pero la proporción de mano dentro del cuadro cambia claramente cómo se comporta el modelo, y es la variable de captura con el efecto más visible de las que pudimos medir.

La iluminación también difiere entre los dos conjuntos, uno más claro y otro notablemente más oscuro y con menos contraste, pero con 22 fotos y dos escenarios no se puede separar su efecto del efecto del encuadre, porque las dos cosas cambian juntas. Lo mismo pasa con el tono de piel y el tamaño de mano: son dos manos distintas, y el dataset original tampoco documenta de dónde salieron las manos que aparecen en sus fotos. Las cuatro letras fotografiadas en los dos conjuntos, A, M, O y V, fallaron en las ocho fotos con los dos modelos, así que ninguna condición rescata a la otra, solo cambia el tipo de error. Con este tamaño de muestra no podemos afirmar que el modelo responda distinto según tono de piel o tamaño de mano, pero sí que es tan sensible a variables de captura que nadie controló al armar el dataset que sería raro que estas otras no lo afectaran.

Para llevar este prototipo a un caso de uso real hace falta bastante más que ajustar hiperparámetros. Se necesitan datos recolectados en condiciones variadas a propósito, con distintas cámaras, distancias, fondos, iluminaciones, tonos de piel y tamaños de mano, en vez de una sola sesión de fotos con una cámara fija. También hace falta una forma de medir qué tan bien funciona el modelo fuera del set de prueba original antes de confiar en el número de accuracy que da ese set, porque como se ve en el ejercicio 8 ese número por sí solo puede ser engañoso.

Como recomendación concreta, antes de clasificar la seña habría que separar la mano del fondo con algún método de segmentación o detección de mano, por ejemplo un modelo preentrenado

que recorte solo la región de la mano antes de pasarla al clasificador. Si el clasificador nunca ve el fondo, no puede depender de él, que es justo lo que le pasó al Random Forest con nuestras fotos. Esto además resolvería buena parte del problema de encuadre y distancia, porque no importaría qué tan lejos esté la cámara si el recorte siempre aísla la mano antes de reducirla a 64x64.

Comparación de algoritmos y selección de modelo final

Para definir cuál modelo es el más adecuado para SignBridge, cruzamos los datos de rendimiento en el set de prueba del dataset con los resultados de la prueba de señas propias.

Los porcentajes de acierto en el set de prueba original son:

Modelo	Accuracy en set de prueba
Random Forest ajustado (300 árboles)	0.9598
Random Forest con aumento	0.9598
CNN base con aumento	0.9594
Random Forest base (200 árboles)	0.9582
CNN base sin aumento	0.9456
CNN profunda	0.7835
Red totalmente conectada (sin aumento)	0.6349
Red totalmente conectada (con aumento)	0.6345

Si nos guiamos solo por el set de prueba, el Random Forest parece la mejor opción. Sin embargo, ese número está inflado porque el modelo aprendió a apoyarse en el fondo estático de la cámara del dataset original. En un producto real, los usuarios tendrán fondos distintos, y es ahí donde el Random Forest se quiebra, como evidenciamos en las pruebas con nuestras propias fotos.

La Red Neuronal Convolutiva (CNN) base con aumento de imágenes es nuestra elección definitiva. Aunque queda ligerísimamente por debajo en el set de prueba, la CNN aprendió a identificar la estructura espacial de la seña, ignorando el fondo. El aumento de imágenes le dio la robustez necesaria para tolerar diferentes condiciones de luz y distancias, logrando un reconocimiento consistente en las fotos nuevas y validando su utilidad para el uso en el mundo real.

Conclusión del proyecto

Este notebook recoge el proceso completo de construcción de nuestro modelo. Empezamos con un análisis exploratorio que nos mostró la consistencia del dataset, preprocesamos las imágenes para estandarizarlas, y evaluamos tres arquitecturas de clasificación: redes totalmente conectadas, redes convolucionales y bosques aleatorios.

Implementar técnicas de aumento de imágenes (data augmentation) probó ser una pieza clave para evitar el sobreajuste y generalizar el aprendizaje. Comprobar los modelos con nuestras propias fotos nos dio la perspectiva necesaria para no quedarnos únicamente con las métricas estáticas del dataset. Al final, la CNN base demostró ser la alternativa más práctica y robusta.

El modelo resultante, junto con las consideraciones de accesibilidad identificadas, nos deja una base sólida para llevar la traducción de lenguaje de señas a dispositivos móviles en un futuro.